ORIS - Aplicação dos 4 requisitos do professor

Guia passo a passo e explicação do código, trecho por trecho

Projeto ORIS (Flask + SQLAlchemy), CESAR School. Pasta do projeto: C:\ORIS\oris. Os trechos de código deste PDF foram extraídos automaticamente dos arquivos reais do projeto.

Como ler este documento

• Capítulo 1: resumo dos 4 requisitos e onde cada um está.

• Capítulos 2 a 5: um capítulo por requisito, com a explicação **do que foi feito, como e por quê**, seguido do código de cada arquivo.

• Capítulo 6: correções feitas após revisão de segurança.

• Capítulo 7: testes. Capítulo 8: roteiro de demonstração para apresentar ao professor.

1. Resumo: o que o professor pediu e onde está

#	Pedido	Onde está	Situação
1	Analisar o contexto e listar papéis e recursos; arquitetura conceitual	docs/ARQUITETURA.md	Novo
2	Gestão de acesso baseada em RBAC	app/utils/decorators.py; PerfilUsuario em app/models/enums.py; todas as rotas	Já existia; reaproveitado e documentado
3	Módulo que consome credenciais LDAP e OIDC e popula as tabelas locais de usuários e grupos	app/services/ldap_service.py, oidc_service.py, diretorio_service.py; app/models/grupo.py; app/routes/auth.py; app/cli.py	Novo
4	Exportar relatórios para armazenamento em nuvem (ex.: Amazon S3) com autenticação avançada	app/services/relatorio_service.py, armazenamento_service.py; app/routes/relatorios.py	Novo

2. Requisito 1 - Papéis, recursos e arquitetura conceitual

O que faz: identifica quem usa o sistema (papéis), o que é protegido (recursos) e como as partes se conectam.

Como funciona: li o enum de perfis e todas as rotas (blueprints), cruzei cada rota com o decorador de acesso que a protege e montei a matriz de permissões e o desenho em camadas. O documento completo está em **docs/ARQUITETURA.md**.

Papéis (perfis)

Papel	Função
ADMINISTRADOR	Acesso total; gerencia usuários
GESTAO_INFORMACAO	Edita, aprova/rejeita alterações, consulta auditoria, importa e exporta
RESPONSAVEL_SAUDE_BUCAL	Cadastra/edita unidades, serviços e equipamentos; importa planilhas
GESTOR	Somente leitura

Recursos

Recurso	Arquivo
Unidades, Serviços, Equipamentos	app/routes/unidades.py, servicos.py, equipamentos.py

Recurso	Arquivo
Alterações (fluxo de aprovação)	app/routes/alteracoes.py
Auditoria	app/routes/auditoria.py
Usuários	app/routes/usuarios.py
Importação de planilhas	app/routes/importacao.py
Relatórios (novo)	app/routes/relatorios.py
Login local e SSO (novo)	app/routes/auth.py

Arquitetura em camadas

```
Navegador --> Rotas Flask (blueprints) + RBAC (roles_required) + auditoria
                |
                +--> Servicos:  diretorio_service (regras)
                |               ldap_service      --> LDAP / Active Directory
                |               oidc_service      --> Provedor OIDC (IdP)
                |               relatorio_service (gera CSV)
                |               armazenamento_service --> Amazon S3
                '--> Modelos SQLAlchemy --> MySQL
                     (usuarios, grupos, usuario_grupos, unidades, ...)
```

Ideia central: as rotas só cuidam de HTTP e permissão; as regras de negócio e as integrações externas ficam em serviços; os dados ficam em modelos. Assim cada integração (LDAP, OIDC, S3) é isolada e pode ser trocada por um falso nos testes.

3. Requisito 2 - RBAC (controle de acesso por papéis)

O que faz: garante que cada usuário só execute o que seu papel permite.

Como funciona: o ORIS já tinha RBAC; ele foi mantido e ampliado. As novas rotas usam os mesmos decoradores e a sincronização LDAP/OIDC converte grupos do diretório em perfis RBAC (capítulo 4). A checagem é sempre feita no backend e devolve HTTP 403; esconder o menu é só conveniência.

Por que assim: RBAC no servidor é o único controle confiável; um botão escondido não impede alguém de chamar a URL.

`app/utils/decorators.py` - a partir da linha 71

```
def roles_required(*perfis_permitidos):
    """Restringe o acesso à rota a usuários autenticados cujo perfil
    esteja entre `perfis_permitidos`.

    Uso:
        @roles_required("ADMINISTRADOR")
        @roles_required("ADMINISTRADOR", "GESTAO_INFORMACAO")

    Fluxo de verificação:
    1. Usuário precisa estar autenticado (sessão válida) - senão,
       redireciona para /login.
    2. O usuário autenticado precisa existir e estar ativo - senão,
       a sessão é encerrada e o usuário é levado de volta ao login
       (trata o caso de um usuário desativado após o login).
    3. O perfil do usuário precisa estar entre os perfis permitidos
       - senão, HTTP 403 (página de acesso negado).
    """

    perfis_permitidos_set = {str(p) for p in perfis_permitidos}

    def decorator(view_func):
        @wraps(view_func)
        def wrapper(*args, **kwargs):
            if not _sessao_autenticada():
                return redirect(url_for("auth.login"))

            usuario = usuario_atual()

            if usuario is None or not usuario.ativo:
                session.clear()
                return redirect(url_for("auth.login"))
```

```
            if usuario.perfil.value not in perfis_permitidos_set:
                abort(403)

            return view_func(*args, **kwargs)

        return wrapper

    return decorator
```

Como ler: *roles_required(*perfis)* devolve um decorador. Antes de executar a rota ele (1) exige sessão autenticada, (2) confere que o usuário ainda está ativo e (3) confere que o perfil dele está na lista permitida; caso contrário responde 403.

Matriz de permissões

Ação	ADMIN	GESTAO	RESP.	GESTOR
Consultar dados	SIM	SIM	SIM	SIM
Criar/editar/alterar situação	SIM	SIM	SIM	-
Aprovar/rejeitar alterações	SIM	SIM	-	-
Consultar auditoria	SIM	SIM	-	-
Gerenciar usuários	SIM	-	-	-
Importar planilhas	SIM	SIM	SIM	-
Baixar CSV de relatório	SIM	SIM	SIM	SIM
Exportar relatório ao S3	SIM	SIM	-	-

Testes do RBAC: tests/test_fase4_rbac.py (já existentes) e os de relatórios em tests/test_fase14_diretorio_relatorios.py.

4. Requisito 3 - LDAP e OIDC populando usuários e grupos

Visão do fluxo

```
LDAP:  flask sincronizar-ldap
        -> ldap_service.buscar_usuarios_ldap()      (le usuarios e grupos)
        -> diretorio_service.sincronizar_usuarios() (grava em usuarios/grupos)

OIDC:  /login/oidc -> provedor (login) -> /login/oidc/callback
        -> oidc_service.concluir_autorizacao()      (valida tokens)
        -> diretorio_service.provisionar_usuario_externo()
        -> sessao aberta no ORIS

Os dois caminhos terminam no MESMO servico de regras (diretorio_service),
entao LDAP e OIDC seguem exatamente a mesma politica de acesso.
```

Passo 1 - Modelo de dados

O que faz: acrescenta ao banco a noção de *de onde vem* o usuário e a tabela de grupos.

Como funciona: o enum *OrigemDiretorio* tem LOCAL, LDAP e OIDC. A tabela *usuarios* ganhou as colunas *origem* e *identificador_externo* (o uid do LDAP ou o *sub* do OIDC). Foram criadas *grupos* (nome + origem, única por par) e *usuario_grupos* (relação muitos-para-muitos).

Por que assim: o identificador externo é estável (e-mail pode mudar); a origem separa contas locais das externas, o que permite as regras de segurança mais adiante.

`app/models/enums.py` - a partir da linha 26

```
class OrigemDiretorio(str, enum.Enum):
    """De onde vem um usuário ou grupo: cadastro local do ORIS ou um
    diretório externo (LDAP / OIDC) - ver app/services/diretorio_service.py."""

    LOCAL = "LOCAL"
    LDAP = "LDAP"
    OIDC = "OIDC"
```

`app/models/grupo.py` - a partir da linha 15

```
usuario_grupos = db.Table(
    "usuario_grupos",
    db.Column("usuario_id", db.Integer, db.ForeignKey("usuarios.id"), primary_key=True),
    db.Column("grupo_id", db.Integer, db.ForeignKey("grupos.id"), primary_key=True),
)


class Grupo(db.Model, TimestampMixin):
    __tablename__ = "grupos"
    __table_args__ = (db.UniqueConstraint("nome", "origem", name="uk_grupos_nome_origem"),)

    id = db.Column(db.Integer, primary_key=True)
    nome = db.Column(db.String(150), nullable=False, index=True)
    origem = db.Column(
        db.Enum(OrigemDiretorio, name="origem_grupo_enum"),
        nullable=False,
        default=OrigemDiretorio.LOCAL,
    )

    usuarios = db.relationship("Usuario", secondary=usuario_grupos, back_populates="grupos")

    def __repr__(self):
        return f"<Grupo id={self.id} nome={self.nome} origem={self.origem}>"
```

`app/models/usuario.py` - a partir da linha 43

```
    # Origem da conta: LOCAL (cadastrada no ORIS, autentica com senha
    # bcrypt) ou LDAP/OIDC (provisionada por app/services/
    # diretorio_service.py; nunca autentica com senha local).
    origem = db.Column(
        db.Enum(OrigemDiretorio, name="origem_usuario_enum"),
        nullable=False,
        default=OrigemDiretorio.LOCAL,
    )

    # Identificador estável no diretório externo (uid do LDAP ou `sub`
```

```
    # do OIDC). Nulo para contas LOCAL.
    identificador_externo = db.Column(db.String(255), nullable=True, index=True)

    # --- Relacionamentos ---
```

Importante: o ORIS **não dá permissão a um grupo diretamente**. A permissão continua sendo o *perfil* do usuário (RBAC). O grupo só serve de entrada para calcular o perfil (Passo 2).

SQL correspondente: **database/schema.sql** (tabelas usuarios, grupos, usuario_grupos) e a migração aditiva **database/migrations/0002_diretorio_ldap_oidc_grupos.sql**, com instruções de rollback no final do arquivo.

Passo 2 - Configuração

O que faz: define, por variáveis de ambiente, como conectar ao LDAP/OIDC e qual grupo vira qual perfil.

Como funciona: tudo é lido de *os.getenv* em config.py; segredos (senha de bind, client secret) ficam no ambiente/.env, nunca no código. O mapa grupo-para-perfil é um JSON.

`config.py` - a partir da linha 62

```
    DIRETORIO_MAPA_GRUPO_PERFIL = os.getenv("DIRETORIO_MAPA_GRUPO_PERFIL", "{}")

    LDAP_URI = os.getenv("LDAP_URI", "")  # ex.: ldaps://ldap.exemplo.com
    LDAP_USE_STARTTLS = os.getenv("LDAP_USE_STARTTLS", "false").lower() == "true"
    LDAP_ALLOW_INSECURE = os.getenv("LDAP_ALLOW_INSECURE", "false").lower() == "true"
    LDAP_BIND_DN = os.getenv("LDAP_BIND_DN", "")
    LDAP_BIND_PASSWORD = os.getenv("LDAP_BIND_PASSWORD", "")
    LDAP_USER_BASE = os.getenv("LDAP_USER_BASE", "")
    LDAP_USER_FILTER = os.getenv("LDAP_USER_FILTER", "(objectClass=person)")
    LDAP_GROUP_BASE = os.getenv("LDAP_GROUP_BASE", "")
    LDAP_GROUP_FILTER = os.getenv("LDAP_GROUP_FILTER", "(objectClass=groupOfNames)")
    LDAP_ATTR_ID = os.getenv("LDAP_ATTR_ID", "uid")
    LDAP_ATTR_NOME = os.getenv("LDAP_ATTR_NOME", "cn")
    LDAP_ATTR_EMAIL = os.getenv("LDAP_ATTR_EMAIL", "mail")
    LDAP_ATTR_GROUP_NAME = os.getenv("LDAP_ATTR_GROUP_NAME", "cn")
    LDAP_ATTR_GROUP_MEMBER = os.getenv("LDAP_ATTR_GROUP_MEMBER", "member")
```

`config.py` - a partir da linha 91

```
    S3_BUCKET = os.getenv("S3_BUCKET", "")
    S3_REGION = os.getenv("S3_REGION", "")
    S3_PREFIX = os.getenv("S3_PREFIX", "oris/relatorios")
    S3_ENDPOINT_URL = os.getenv("S3_ENDPOINT_URL", "")  # opcional (MinIO/LocalStack)
    S3_URL_EXPIRA_SEGUNDOS = int(os.getenv("S3_URL_EXPIRA_SEGUNDOS", "900"))
```

Exemplo de mapa: `{"oris-admin":"ADMINISTRADOR","oris-gestao":"GESTAO_INFORMACAO"}`

Passo 3 - Regras de negócio: diretorio_service.py

Este é o coração do módulo. Ele não conhece LDAP nem OIDC: recebe objetos *UsuarioExterno* e aplica a política.

3.1 O objeto UsuarioExterno e a leitura do mapa

`app/services/diretorio_service.py` - a partir da linha 59

```
@dataclass
class UsuarioExterno:
    identificador: str
    nome: str
    email: str
    grupos: list = field(default_factory=list)
```

`app/services/diretorio_service.py` - a partir da linha 67

```
def carregar_mapa_grupo_perfil(config=None):
    """Lê e valida `DIRETORIO_MAPA_GRUPO_PERFIL` (JSON grupo -> perfil)."""
    config = config if config is not None else current_app.config
    bruto = config.get("DIRETORIO_MAPA_GRUPO_PERFIL") or "{}"
    try:
        mapa = json.loads(bruto) if isinstance(bruto, str) else dict(bruto)
    except ValueError as erro:
        raise DiretorioConfigError("DIRETORIO_MAPA_GRUPO_PERFIL não é um JSON válido.") from erro

    validos = {perfil.value for perfil in PerfilUsuario}
    resultado = {}
    for grupo, perfil in mapa.items():
```

```
        if perfil not in validos:
            raise DiretorioConfigError(f"Perfil '{perfil}' do grupo '{grupo}' não existe no ORIS.")
        resultado[str(grupo).strip().lower()] = PerfilUsuario(perfil)
    return resultado
```

UsuarioExterno é um objeto simples (identificador, nome, e-mail, lista de grupos) que serve de formato comum para LDAP e OIDC. *carregar_mapa_grupo_perfil* lê o JSON, valida que cada perfil existe no ORIS (senão recusa com erro claro) e normaliza os nomes de grupo para minúsculas.

3.2 Escolha do perfil

app/services/diretorio_service.py - a partir da linha 85

```
def resolver_perfil(grupos, mapa):
    """Perfil de maior privilégio entre os grupos mapeados, ou None."""
    encontrados = {mapa[g.strip().lower()] for g in grupos if g.strip().lower() in mapa}
    for perfil in PRIORIDADE_PERFIS:
        if perfil in encontrados:
            return perfil
    return None
```

Se o usuário está em vários grupos mapeados, vale o de **maior privilégio**, seguindo a lista PRIORIDADE_PERFIS (ADMINISTRADOR, GESTAO_INFORMACAO, RESPONSAVEL_SAUDE_BUCAL, GESTOR). Se nenhum grupo está no mapa, devolve *None*: o usuário não recebe acesso.

3.3 Proteção do último administrador

app/services/diretorio_service.py - a partir da linha 94

```
def _e_ultimo_admin_ativo(usuario):
    if usuario.perfil != PerfilUsuario.ADMINISTRADOR or not usuario.ativo:
        return False
    outros = Usuario.query.filter(
        Usuario.perfil == PerfilUsuario.ADMINISTRADOR,
        Usuario.ativo.is_(True),
        Usuario.id != usuario.id,
    ).count()
    return outros == 0
```

Conta quantos *outros* administradores ativos existem. Se não há nenhum, o usuário é o último e a sincronização **não pode** desativá-lo nem rebaixá-lo; assim uma mudança no diretório nunca deixa o sistema sem administrador.

3.4 Sincronização dos grupos do usuário

app/services/diretorio_service.py - a partir da linha 105

```
def _sincronizar_grupos(usuario, nomes_grupos, origem):
    """Substitui a participação do usuário nos grupos DESTA origem
    (grupos de outras origens ficam intactos)."""
    manter = [g for g in usuario.grupos if g.origem != origem]
    for nome in sorted({n.strip() for n in nomes_grupos if n.strip()}):
        grupo = Grupo.query.filter_by(nome=nome, origem=origem).first()
        if grupo is None:
            grupo = Grupo(nome=nome, origem=origem)
            db.session.add(grupo)
        manter.append(grupo)
    usuario.grupos = manter
```

Substitui os grupos do usuário **apenas da origem atual** (grupos LDAP não apagam grupos OIDC). Cria o grupo na tabela *grupos* se ele ainda não existir e ajusta a relação em *usuario_grupos*.

3.5 Aplicar um usuário (_aplicar_usuario)

É a função mais importante. Está dividida abaixo em três partes.

Parte A - localizar a conta existente

app/services/diretorio_service.py (_aplicar_usuario, parte A) - a partir da linha 118

```
def _aplicar_usuario(externo, origem, mapa, resumo):
    email = externo.email.strip().lower()
    if not externo.identificador or not email:
        resumo["ignorados"].append(f"{externo.identificador or '(sem id)'}: sem identificador ou e-mail")
        return None
```

```
    perfil = resolver_perfil(externo.grupos, mapa)

    usuario = Usuario.query.filter_by(origem=origem, identificador_externo=externo.identificador).first()
    if usuario is None:
        por_email = Usuario.query.filter_by(email=email).first()
        if por_email is not None:
            if por_email.origem != origem:
                resumo["ignorados"].append(f"{email}: já existe uma conta {por_email.origem.value} com este
                    e-mail")
                return None
            if por_email.identificador_externo and por_email.identificador_externo != externo.identificador:
                # Conta já vinculada a outra identidade: nunca re-vincular por e-mail.
                resumo["ignorados"].append(f"{email}: e-mail já vinculado a outro identificador externo")
                return None
            usuario = por_email
            usuario.identificador_externo = externo.identificador
```

• Descarta entradas sem identificador ou sem e-mail.

• Calcula o perfil pelos grupos.

• Procura primeiro pelo par estável (origem + identificador_externo).

• Se não achou, procura por e-mail, mas com três travas: (1) se existe conta de **outra origem** (por exemplo LOCAL) com esse e-mail, ignora e relata, assim o diretório nunca assume uma conta local; (2) se a conta já está vinculada a **outro identificador externo**, ignora (evita sequestro de conta por e-mail repetido; ver capítulo 6); (3) só então vincula.

Parte B - criar usuário novo

app/services/diretorio_service.py (_aplicar_usuario, parte B) - a partir da linha 140

```
    if usuario is None:
        if perfil is None:
            resumo["ignorados"].append(f"{email}: nenhum grupo mapeado para um perfil")
            return None
        usuario = Usuario(
            nome=externo.nome.strip() or email,
            email=email,
            senha_hash=gerar_hash_senha(secrets.token_urlsafe(32)),
            perfil=perfil,
            ativo=True,
            origem=origem,
            identificador_externo=externo.identificador,
        )
        db.session.add(usuario)
        db.session.flush()
        _sincronizar_grupos(usuario, externo.grupos, origem)
        registrar_auditoria(
            usuario=usuario,
            acao="CRIAR_USUARIO",
            tabela="usuarios",
            registro_id=usuario.id,
            descricao=f"Usuário '{email}' provisionado via {origem.value}, perfil {perfil.value}.",
        )
        resumo["criados"] += 1
        return usuario
```

• Sem grupo mapeado: **não cria** (estar no diretório não basta).

• A senha é um valor aleatório descartável (*secrets.token_urlsafe(32)*) já em hash bcrypt: ninguém a conhece e, de todo modo, contas externas não entram por senha local.

• Cria o usuário, sincroniza grupos e registra auditoria *CRIAR_USUARIO*.

Parte C - atualizar usuário existente

app/services/diretorio_service.py (_aplicar_usuario, parte C) - a partir da linha 166

```
    alterou = False
    nome_novo = externo.nome.strip() or usuario.nome
    if usuario.nome != nome_novo:
        usuario.nome = nome_novo
        alterou = True
    if usuario.email != email:
        usuario.email = email
        alterou = True
```

```
    if perfil is None:
        if usuario.ativo:
            if _e_ultimo_admin_ativo(usuario):
                resumo["protegidos"].append(f"{usuario.email}: único ADMINISTRADOR ativo, não desativado")
            else:
                usuario.ativo = False
                registrar_auditoria(
                    usuario=usuario, acao="DESATIVAR_USUARIO", tabela="usuarios", registro_id=usuario.id,
                    descricao=f"Desativação de '{usuario.email}' via {origem.value}: sem grupo mapeado.",
                )
                resumo["desativados"] += 1
    else:
        if perfil != usuario.perfil:
            if _e_ultimo_admin_ativo(usuario):
                resumo["protegidos"].append(f"{usuario.email}: único ADMINISTRADOR ativo, perfil mantido")
            else:
                anterior = usuario.perfil
                usuario.perfil = perfil
                registrar_auditoria(
                    usuario=usuario, acao="ALTERAR_PERFIL", tabela="usuarios", registro_id=usuario.id,
                    descricao=f"Perfil de '{usuario.email}' alterado via {origem.value}.",
                    valor_anterior={"perfil": anterior.value}, valor_novo={"perfil": perfil.value},
                )
                alterou = True
        if not usuario.ativo:
            usuario.ativo = True
            registrar_auditoria(
                usuario=usuario, acao="ATIVAR_USUARIO", tabela="usuarios", registro_id=usuario.id,
                descricao=f"Reativação de '{usuario.email}' via {origem.value}.",
            )
            alterou = True

    _sincronizar_grupos(usuario, externo.grupos, origem)
    if alterou:
        resumo["atualizados"] += 1
    return usuario
```

• Atualiza nome e e-mail.

• Sem grupo mapeado: desativa (exceto o último admin, que vai para a lista de "protegidos"), com auditoria.

• Com grupo: se o perfil mudou, altera e audita (*ALTERAR_PERFIL*, guardando valor anterior e novo), também respeitando o último admin; reativa se estava inativo (*ATIVAR_USUARIO*).

• No fim, sincroniza os grupos e contabiliza se houve alteração.

3.6 Sincronização em lote

app/services/diretorio_service.py - a partir da linha 213

```
def sincronizar_usuarios(externos, origem, desativar_ausentes=True, mapa=None, config=None, simular=False):
    """Aplica a lista de `UsuarioExterno` às tabelas locais, numa única
    transação. Retorna um resumo (criados/atualizados/desativados e
    listas de ignorados/protegidos). Faz commit; em erro, rollback.
    Com `simular=True` calcula o resumo e desfaz tudo (rollback)."""
    if mapa is None:
        mapa = carregar_mapa_grupo_perfil(config)
    if not mapa:
        raise DiretorioConfigError("DIRETORIO_MAPA_GRUPO_PERFIL está vazio: nenhum usuário poderia receber um
            perfil.")

    resumo = {"criados": 0, "atualizados": 0, "desativados": 0, "ignorados": [], "protegidos": []}
    try:
        vistos = set()
        for externo in externos:
            usuario = _aplicar_usuario(externo, origem, mapa, resumo)
            if usuario is not None:
                vistos.add(usuario.id)
            vistos.add(("ext", externo.identificador))

        if desativar_ausentes:
            ativos = Usuario.query.filter_by(origem=origem, ativo=True).all()
            for usuario in ativos:
                if ("ext", usuario.identificador_externo) in vistos:
                    continue
                if _e_ultimo_admin_ativo(usuario):
                    resumo["protegidos"].append(f"{usuario.email}: único ADMINISTRADOR ativo, não desativado")
                    continue
                usuario.ativo = False
                registrar_auditoria(
```

```
                    usuario=usuario, acao="DESATIVAR_USUARIO", tabela="usuarios", registro_id=usuario.id,
                    descricao=f"Desativação de '{usuario.email}' via {origem.value}: ausente no diretório.",
                )
                resumo["desativados"] += 1

        if simular:
            db.session.rollback()
        else:
            db.session.commit()
    except Exception:
        db.session.rollback()
        raise
    return resumo
```

• Valida o mapa (vazio é recusado: ninguém poderia receber perfil).

• Aplica cada usuário externo; guarda os que foram vistos.

• Se *desativar_ausentes*: quem é da origem, está ativo e não apareceu no diretório é desativado (de novo respeitando o último admin) com auditoria.

• **Uma única transação**: tudo grava junto (commit) ou nada grava (rollback em erro).

• Com *simular=True* calcula o resumo e faz rollback: serve de "ensaio" sem gravar.

3.7 Provisionar um usuário no login OIDC

app/services/diretorio_service.py - a partir da linha 257

```
def provisionar_usuario_externo(externo, origem, config=None):
    """Provisiona UM usuário (login OIDC). Devolve o Usuario ativo, ou
    None se ele não tem grupo mapeado / está bloqueado."""
    mapa = carregar_mapa_grupo_perfil(config)
    resumo = sincronizar_usuarios([externo], origem, desativar_ausentes=False, mapa=mapa or None, config=config)
    usuario = Usuario.query.filter_by(origem=origem, identificador_externo=externo.identificador).first()
    if usuario is None or not usuario.ativo:
        return None
    return usuario
```

Reaproveita a mesma sincronização para um único usuário (sem desativar os demais) e só devolve o usuário se ele existir e estiver ativo; caso contrário devolve *None* e o login é negado.

Passo 4 - Leitura do LDAP: ldap_service.py

4.1 Abrir a conexão com segurança

app/services/ldap_service.py - a partir da linha 22

```
def _abrir_conexao(config):
    """Abre a conexão autenticada com a conta de serviço."""
    from ldap3 import ALL, Connection, Server

    uri = config.get("LDAP_URI") or ""
    if not uri:
        raise DiretorioConfigError("LDAP_URI não configurado.")

    usa_tls = uri.lower().startswith("ldaps://") or config.get("LDAP_USE_STARTTLS")
    if not usa_tls and not config.get("LDAP_ALLOW_INSECURE"):
        raise DiretorioConfigError(
            "Conexão LDAP sem TLS recusada: use ldaps:// ou LDAP_USE_STARTTLS=true "
            "(ou LDAP_ALLOW_INSECURE=true apenas em laboratório)."
        )

    servidor = Server(uri, get_info=ALL, connect_timeout=10)
    conexao = Connection(
        servidor,
        user=config.get("LDAP_BIND_DN") or None,
        password=config.get("LDAP_BIND_PASSWORD") or None,
        receive_timeout=20,
    )
    if config.get("LDAP_USE_STARTTLS") and not uri.lower().startswith("ldaps://"):
        conexao.open()
        conexao.start_tls()
    if not conexao.bind():
        raise DiretorioConfigError("Falha ao autenticar a conta de serviço no LDAP.")
    return conexao
```

• Sem *LDAP_URI*: erro de configuração.

• **TLS obrigatório**: exige *ldaps://* ou STARTTLS; conexão em texto puro só com *LDAP_ALLOW_INSECURE=true* (laboratório), pois a senha da conta de serviço trafegaria aberta.

• Usa timeouts (10 s para conectar, 20 s para receber) para não travar o sistema.

• Faz o *bind* com a conta de serviço (DN e senha vêm do ambiente). Se falhar, erro claro.

4.2 Buscar usuários e grupos

app/services/ldap_service.py - a partir da linha 59

```
def buscar_usuarios_ldap(config=None, conexao=None):
    """Devolve a lista de `UsuarioExterno` do diretório. `conexao`
    permite injetar uma conexão já aberta (testes usam o modo MOCK do
    próprio ldap3)."""
    config = config if config is not None else current_app.config
    propria = conexao is None
    if propria:
        conexao = _abrir_conexao(config)

    try:
        attr_id = config.get("LDAP_ATTR_ID", "uid")
        attr_nome = config.get("LDAP_ATTR_NOME", "cn")
        attr_email = config.get("LDAP_ATTR_EMAIL", "mail")
        attr_grupo_nome = config.get("LDAP_ATTR_GROUP_NAME", "cn")
        attr_membros = config.get("LDAP_ATTR_GROUP_MEMBER", "member")

        # 1) Grupos: DN do usuário -> nomes dos grupos.
        grupos_por_dn = {}
        base_grupos = config.get("LDAP_GROUP_BASE")
        if base_grupos:
            conexao.search(
                base_grupos,
                config.get("LDAP_GROUP_FILTER", "(objectClass=groupOfNames)"),
                attributes=[attr_grupo_nome, attr_membros],
            )
            for grupo in conexao.entries:
                nomes = _valores(grupo, attr_grupo_nome)
                if not nomes:
                    continue
                for dn_membro in _valores(grupo, attr_membros):
                    grupos_por_dn.setdefault(dn_membro.strip().lower(), set()).add(nomes[0])

        # 2) Usuários.
        conexao.search(
            config.get("LDAP_USER_BASE", ""),
            config.get("LDAP_USER_FILTER", "(objectClass=person)"),
            attributes=[attr_id, attr_nome, attr_email, "memberOf"],
        )
        externos = []
        for entrada in conexao.entries:
            ids = _valores(entrada, attr_id)
            if not ids:
                continue
            grupos = set(grupos_por_dn.get(str(entrada.entry_dn).strip().lower(), set()))
            for dn_grupo in _valores(entrada, "memberOf"):
                # "CN=oris-admin,OU=Grupos,..." -> "oris-admin"
                primeiro = dn_grupo.split(",")[0]
                if "=" in primeiro:
                    grupos.add(primeiro.split("=", 1)[1])
            nomes = _valores(entrada, attr_nome)
            emails = _valores(entrada, attr_email)
            externos.append(
                UsuarioExterno(
                    identificador=ids[0],
                    nome=nomes[0] if nomes else ids[0],
                    email=emails[0] if emails else "",
                    grupos=sorted(grupos),
                )
            )
        return externos
    finally:
        if propria:
            conexao.unbind()
```

• **Passo 1:** pesquisa os grupos na base de grupos e monta um dicionário DN do membro para nomes de grupos.

• **Passo 2:** pesquisa os usuários; para cada um pega id, nome e e-mail e junta os grupos do dicionário com os do atributo *memberOf* (se existir).

• Devolve uma lista de *UsuarioExterno*. A conexão é fechada no *finally* se foi aberta aqui.

• O parâmetro *conexao* permite injetar uma conexão pronta: os testes usam o modo MOCK do ldap3 (LDAP falso em memória).

Passo 5 - Comando de linha: flask sincronizar-ldap

app/cli.py - a partir da linha 60

```
@click.command("sincronizar-ldap")
@click.option("--simular", is_flag=True, help="Mostra o que mudaria, sem gravar nada.")
@click.option("--manter-ausentes", is_flag=True, help="Não desativa usuários LDAP que sumiram do diretório.")
@with_appcontext
def sincronizar_ldap_command(simular, manter_ausentes):
    """Popula usuários e grupos locais a partir do diretório LDAP (Requisito 3)."""
    from app.models import OrigemDiretorio
    from app.services.diretorio_service import DiretorioConfigError, sincronizar_usuarios
    from app.services.ldap_service import buscar_usuarios_ldap

    try:
        externos = buscar_usuarios_ldap()
        resumo = sincronizar_usuarios(
            externos, OrigemDiretorio.LDAP, desativar_ausentes=not manter_ausentes, simular=simular
        )
    except DiretorioConfigError as erro:
        click.echo(click.style(f"Configuração inválida: {erro}", fg="red"))
        raise SystemExit(1)

    prefixo = "[SIMULAÇÃO] " if simular else ""
    click.echo(
        click.style(
            f"{prefixo}{len(externos)} usuário(s) lido(s) do LDAP - criados: {resumo['criados']}, "
            f"atualizados: {resumo['atualizados']}, desativados: {resumo['desativados']}.",
            fg="green",
        )
    )
    for motivo in resumo["ignorados"]:
        click.echo(f"  ignorado: {motivo}")
    for motivo in resumo["protegidos"]:
        click.echo(click.style(f"  protegido: {motivo}", fg="yellow"))
```

• `flask sincronizar-ldap` lê o LDAP e grava; `--simular` mostra o que mudaria sem gravar; `--manter-ausentes` não desativa quem sumiu.

• Imprime o resumo (criados, atualizados, desativados), os ignorados e os protegidos. Erro de configuração sai com código 1.

• Registrado em *register_cli_commands* no mesmo arquivo.

Passo 6 - Login OIDC

6.1 Montar o pedido ao provedor (iniciar_autorizacao)

app/services/oidc_service.py - a partir da linha 79

```
def iniciar_autorizacao(redirect_uri, config=None):
    """Devolve (url_de_autorizacao, estado). `estado` deve ser guardado
    na sessão até o callback."""
    config = config if config is not None else current_app.config
    metadados = _descobrir(config)

    estado = {
        "state": secrets.token_urlsafe(24),
        "nonce": secrets.token_urlsafe(24),
        "code_verifier": secrets.token_urlsafe(48),
    }
    desafio = _b64url(hashlib.sha256(estado["code_verifier"].encode("ascii")).digest())
    parametros = {
        "response_type": "code",
        "client_id": config["OIDC_CLIENT_ID"],
        "redirect_uri": redirect_uri,
        "scope": config.get("OIDC_SCOPES", "openid profile email"),
        "state": estado["state"],
```

```
        "nonce": estado["nonce"],
        "code_challenge": desafio,
        "code_challenge_method": "S256",
    }
    url = f"{metadados['authorization_endpoint']}?{urllib.parse.urlencode(parametros)}"
    return url, estado
```

• Descobre os endereços do provedor em */.well-known/openid-configuration* (recusa http:// sem a opção de laboratório).

• Gera três valores aleatórios: **state** (anti-CSRF), **nonce** (anti-replay do token) e **code_verifier** (PKCE).

• Calcula o **code_challenge** = SHA-256 do verifier em base64url (método S256) e monta a URL de autorização. O verifier nunca vai na URL; só o desafio.

• Devolve a URL e o "estado" a guardar na sessão do navegador.

6.2 Concluir o login (concluir_autorizacao)

app/services/oidc_service.py - a partir da linha 114

```
def concluir_autorizacao(codigo, estado, redirect_uri, config=None):
    """Troca o `code` por tokens, valida o id_token e devolve um
    `UsuarioExterno` com as claims do usuário."""
    config = config if config is not None else current_app.config
    metadados = _descobrir(config)
    _exigir_https(metadados["token_endpoint"], config)

    tokens = _http_post_form(
        metadados["token_endpoint"],
        {
            "grant_type": "authorization_code",
            "code": codigo,
            "redirect_uri": redirect_uri,
            "client_id": config["OIDC_CLIENT_ID"],
            "client_secret": config.get("OIDC_CLIENT_SECRET", ""),
            "code_verifier": estado["code_verifier"],
        },
    )

    id_token = tokens.get("id_token")
    if not id_token:
        raise OidcError("Resposta sem id_token.")
    claims = _payload_jwt(id_token)

    if claims.get("iss", "").rstrip("/") != config["OIDC_ISSUER"].rstrip("/"):
        raise OidcError("Emissor (iss) do id_token inesperado.")
    audiencia = claims.get("aud")
    audiencias = audiencia if isinstance(audiencia, list) else [audiencia]
    if config["OIDC_CLIENT_ID"] not in audiencias:
        raise OidcError("Audiência (aud) do id_token inesperada.")
    if not secrets.compare_digest(str(claims.get("nonce", "")), estado["nonce"]):
        raise OidcError("nonce do id_token não confere.")

    if tokens.get("access_token") and metadados.get("userinfo_endpoint"):
        _exigir_https(metadados["userinfo_endpoint"], config)
        info = _http_get_json(
            metadados["userinfo_endpoint"], {"Authorization": f"Bearer {tokens['access_token']}"}
        )
        if info.get("sub") != claims.get("sub"):
            raise OidcError("sub do userinfo difere do id_token.")
        claims = {**claims, **info}

    if claims.get("email_verified") is not True:
        raise OidcError("E-mail não verificado no provedor.")

    grupos = claims.get(config.get("OIDC_CLAIM_GROUPS", "groups"), []) or []
    if isinstance(grupos, str):
        grupos = [grupos]

    return UsuarioExterno(
        identificador=str(claims.get("sub", "")),
        nome=claims.get("name") or claims.get("preferred_username") or "",
        email=claims.get("email", ""),
        grupos=[str(g) for g in grupos],
    )
```

• Troca o *code* por tokens no endpoint de token (servidor a servidor, por HTTPS), enviando o *code_verifier* do PKCE.

• Lê o id_token e valida: **iss** (emissor esperado), **aud** (contém o nosso client_id) e **nonce** (igual ao que enviamos, comparado em tempo constante).

• Busca o *userinfo* e confere que o *sub* é o mesmo do id_token (impede mistura de identidades).

• Exige **email_verified == true** (ver capítulo 6).

• Lê o claim de grupos (nome configurável) e devolve um *UsuarioExterno*.

• Sobre a assinatura do id_token: como ele chega direto do provedor por TLS no canal de token, a validação de assinatura é dispensada pela especificação (OIDC Core 3.1.3.7).

6.3 As rotas no Flask

app/routes/auth.py - a partir da linha 120

```
@auth_bp.route("/login/oidc")
def login_oidc():
    if not oidc_habilitado():
        abort(404)
    try:
        url, estado = iniciar_autorizacao(_uri_callback_oidc())
    except Exception:
        current_app.logger.exception("Falha ao iniciar o login OIDC")
        flash(MENSAGEM_SSO_NEGADO, "danger")
        return redirect(url_for("auth.login"))
    session["oidc_estado"] = estado
    return redirect(url)
```

app/routes/auth.py - a partir da linha 134

```
@auth_bp.route("/login/oidc/callback")
def login_oidc_callback():
    if not oidc_habilitado():
        abort(404)

    estado = session.pop("oidc_estado", None)
    state_recebido = request.args.get("state", "")
    codigo = request.args.get("code", "")
    if (
        not estado
        or not codigo
        or not secrets.compare_digest(state_recebido.encode("utf-8"), estado["state"].encode("utf-8"))
    ):
        flash(MENSAGEM_SSO_NEGADO, "danger")
        return redirect(url_for("auth.login"))

    try:
        externo = concluir_autorizacao(codigo, estado, _uri_callback_oidc())
        usuario = provisionar_usuario_externo(externo, OrigemDiretorio.OIDC)
    except (OidcError, DiretorioConfigError, OSError, ValueError, KeyError):
        current_app.logger.exception("Falha no callback OIDC")
        db.session.rollback()
        flash(MENSAGEM_SSO_NEGADO, "danger")
        return redirect(url_for("auth.login"))

    if usuario is None:
        flash(MENSAGEM_SSO_NEGADO, "danger")
        return redirect(url_for("auth.login"))

    session.clear()
    session["usuario_id"] = usuario.id
    session["autenticado"] = True
    session.permanent = True
    registrar_auditoria(usuario=usuario, acao="LOGIN", descricao="Login via OIDC.")
    db.session.commit()
    return redirect(url_for("main.index"))
```

• */login/oidc*: se o OIDC está desligado, 404. Senão guarda o estado na sessão e redireciona ao provedor. Erros viram mensagem genérica.

• */login/oidc/callback*: retira o estado da sessão (uso único), confere o *state* recebido (comparação em tempo constante), conclui a autorização e provisiona o usuário. Qualquer falha mostra a mesma mensagem genérica

(sem vazar detalhes) e registra o erro no log.

• Sucesso: *session.clear()* (evita session fixation), grava o usuário na sessão, audita "Login via OIDC".

6.4 Senha local só para contas LOCAL

app/routes/auth.py - a partir da linha 81

```
        login_valido = (
            usuario is not None
            and usuario.ativo
            and usuario.origem == OrigemDiretorio.LOCAL
            and verificar_senha(senha, usuario.senha_hash)
        )
```

A autenticação por senha só vale para contas de origem LOCAL. Uma conta LDAP/OIDC nunca entra com a senha aleatória do banco; a autenticação dela é sempre feita no diretório.

6.5 O botão de SSO na tela de login

app/templates/login.html - a partir da linha 60

```
            {% if config.get('OIDC_ENABLED') and config.get('OIDC_ISSUER') %}
            <a href="{{ url_for('auth.login_oidc') }}" class="btn btn-outline-secondary w-100 mt-3">
                <i class="bi bi-box-arrow-in-right" aria-hidden="true"></i> Entrar com login institucional (SSO)
            </a>
            {% endif %}
```

O botão só aparece se o OIDC está habilitado e configurado.

Resumo das regras de segurança do Requisito 3

Regra	Onde
Conta externa não entra com senha local	auth.py (login_valido)
Sincronização nunca altera conta LOCAL	diretorio_service._aplicar_usuario (parte A)
Sem grupo mapeado = sem acesso	resolver_perfil / partes B e C
Último administrador protegido	_e_ultimo_admin_ativo
Transação única + modo simulação	sincronizar_usuarios
TLS no LDAP, HTTPS no OIDC	_abrir_conexao, _exigir_https
PKCE, state, nonce, iss, aud, sub, email_verified	oidc_service, auth.py
Mensagens de erro genéricas ao usuário	login_oidc_callback
Auditoria de toda mudança	registrar_auditoria em cada ação

5. Requisito 4 - Exportar relatórios para a nuvem (Amazon S3)

Visão do fluxo

```
Usuario (ADMIN ou GESTAO_INFORMACAO) clica em 'Exportar para S3'
   -> POST /relatorios/<entidade>/exportar-s3     (relatorios.py, RBAC)
   -> relatorio_service.gerar_csv()               (gera o CSV, sanitiza celulas)
   -> armazenamento_service.enviar_relatorio()    (put_object com criptografia)
   -> armazenamento_service.gerar_url_download()  (link temporario)
   -> auditoria EXPORTAR_RELATORIO
```

Passo 1 - Gerar os relatórios (relatorio_service.py)

1.1 Proteção contra formula injection

app/services/relatorio_service.py - a partir da linha 25

```
def sanitizar_celula(valor):
    if valor is None:
        return ""
    if isinstance(valor, bool):
        return "SIM" if valor else "NAO"
    if hasattr(valor, "value"):  # Enum
        valor = valor.value
    texto = str(valor)
    if texto.startswith(_PERIGOSOS):
        return "'" + texto
    return texto
```

Planilhas (Excel, Sheets) executam como fórmula qualquer célula começada por **= + -** @, TAB ou CR. Como os dados do ORIS podem vir de planilhas importadas, um valor malicioso como `=HYPERLINK(...)` poderia ser executado por quem abrir o relatório. A função coloca um apóstrofo na frente, o que força o programa a tratar a célula como texto. Também converte Enum em texto e booleano em SIM/NAO. Isso fecha o item SEC-008 do documento de segurança.

1.2 Montar as linhas de cada relatório

app/services/relatorio_service.py - a partir da linha 38

```
def _linhas_unidades():
    cabecalho = ["id", "nome", "cnes", "unidade_mista", "tipo", "endereco", "bairro", "cidade", "uf",
        "situacao"]
    linhas = [
        [u.id, u.nome, u.cnes, u.unidade_e_mista, u.tipo, u.endereco, u.bairro, u.cidade, u.uf, u.situacao]
        for u in Unidade.query.order_by(Unidade.nome).all()
    ]
    return cabecalho, linhas
```

Cada relatório tem uma função que consulta o banco via ORM (sem SQL manual) e devolve cabeçalho e linhas; existem uma para unidades, uma para serviços e uma para equipamentos (esta traz também o nome da unidade e do serviço relacionados).

1.3 Gerar o arquivo CSV

app/services/relatorio_service.py - a partir da linha 75

```
def gerar_csv(entidade):
    """Devolve (nome_do_arquivo, bytes). `entidade` precisa estar em
    ENTIDADES_RELATORIO (quem chama valida; aqui um valor inválido
    levanta ValueError)."""
    if entidade not in _GERADORES:
        raise ValueError("Entidade de relatório desconhecida.")

    cabecalho, linhas = _GERADORES[entidade]()
    buffer = io.StringIO()
    escritor = csv.writer(buffer)
    escritor.writerow(cabecalho)
    for linha in linhas:
        escritor.writerow([sanitizar_celula(c) for c in linha])

    carimbo = utcnow().strftime("%Y%m%d-%H%M%S")
    nome_arquivo = f"oris-{entidade}-{carimbo}.csv"
```

```
    return nome_arquivo, ("?" + buffer.getvalue()).encode("utf-8")
```

Escolhe o gerador pela entidade (valor fora da lista fixa levanta erro), escreve o CSV passando cada célula pela sanitização, e devolve o nome do arquivo (*oris-unidades-AAAAMMDD-HHMMSS.csv*) e os bytes em UTF-8 com BOM para abrir com acentos no Excel.

Passo 2 - Enviar ao S3 (armazenamento_service.py)

2.1 Autenticação avançada: o cliente S3

app/services/armazenamento_service.py - a partir da linha 37

```
def _criar_cliente(config):
    import boto3

    opcoes = {}
    if config.get("S3_REGION"):
        opcoes["region_name"] = config["S3_REGION"]
    if config.get("S3_ENDPOINT_URL"):
        opcoes["endpoint_url"] = config["S3_ENDPOINT_URL"]
    return boto3.client("s3", **opcoes)
```

Repare que **não há chave de acesso em nenhum lugar**. O *boto3.client('s3')* procura credenciais sozinho pela cadeia padrão da AWS: variáveis de ambiente AWS_ACCESS_KEY_ID/AWS_SECRET_ACCESS_KEY, arquivo de perfil ~/.aws ou, em produção, um **IAM Role** da máquina (credenciais temporárias e rotacionadas). Cada requisição é assinada com **AWS Signature V4** pelo próprio boto3. Isso é mais seguro do que guardar segredo no projeto.

2.2 Nome (chave) do objeto no bucket

app/services/armazenamento_service.py - a partir da linha 48

```
def montar_chave(entidade, config=None):
    config = config if config is not None else current_app.config
    prefixo = (config.get("S3_PREFIX") or "").strip("/")
    carimbo = utcnow().strftime("%Y/%m/%d")
    nome = f"{entidade}-{utcnow().strftime('%H%M%S')}-{uuid.uuid4().hex[:12]}.csv"
    return "/".join(parte for parte in (prefixo, entidade, carimbo, nome) if parte)
```

A chave é montada só com valores controlados pelo sistema: prefixo de configuração + entidade (de uma lista fixa) + data + hora + 12 caracteres aleatórios (UUID). Nenhum texto digitado pelo usuário entra no caminho, o que elimina ataques de *path traversal* e sobrescrita de arquivos.

2.3 Enviar o arquivo

app/services/armazenamento_service.py - a partir da linha 56

```
def enviar_relatorio(conteudo, entidade, cliente=None, config=None):
    """Envia o CSV ao bucket e devolve a chave do objeto. `cliente`
    permite injetar um cliente S3 (testes usam um falso)."""
    config = config if config is not None else current_app.config
    if not s3_configurado(config):
        raise ArmazenamentoError("S3_BUCKET não configurado.")

    chave = montar_chave(entidade, config)
    try:
        cliente = cliente or _criar_cliente(config)
        cliente.put_object(
            Bucket=config["S3_BUCKET"],
            Key=chave,
            Body=conteudo,
            ContentType="text/csv; charset=utf-8",
            ServerSideEncryption="AES256",
        )
    except Exception as erro:
        current_app.logger.exception("Falha ao enviar relatório ao S3")
        raise ArmazenamentoError("Não foi possível enviar o relatório ao armazenamento.") from erro
    return chave
```

• Sem bucket configurado: erro (a exportação fica desativada).

• *put_object* com **ServerSideEncryption="AES256"**: o arquivo fica criptografado em repouso na AWS.

• Qualquer falha é registrada no log (*logger.exception*) e convertida em *ArmazenamentoError* com mensagem genérica: detalhes da AWS (nomes de bucket, contas) nunca chegam ao usuário.

• O parâmetro *cliente* permite injetar um cliente falso nos testes.

2.4 Link de download temporário

app/services/armazenamento_service.py - a partir da linha 79

```
def gerar_url_download(chave, cliente=None, config=None):
    """URL pré-assinada (GET) de curta duração para o objeto."""
    config = config if config is not None else current_app.config
    try:
        cliente = cliente or _criar_cliente(config)
        return cliente.generate_presigned_url(
            "get_object",
            Params={"Bucket": config["S3_BUCKET"], "Key": chave},
            ExpiresIn=int(config.get("S3_URL_EXPIRA_SEGUNDOS", 900)),
        )
    except Exception as erro:
        current_app.logger.exception("Falha ao gerar URL pré-assinada")
        raise ArmazenamentoError("Não foi possível gerar o link de download.") from erro
```

Gera uma **URL pré-assinada** com validade curta (padrão 900 s = 15 min). O bucket permanece privado; quem tem o link só consegue baixar aquele objeto e só por pouco tempo.

Passo 3 - Rotas e RBAC (app/routes/relatorios.py)

app/routes/relatorios.py - a partir da linha 31

```
PERFIS_QUE_EXPORTAM = (
    PerfilUsuario.ADMINISTRADOR.value,
    PerfilUsuario.GESTAO_INFORMACAO.value,
)
```

app/routes/relatorios.py - a partir da linha 57

```
@relatorios_bp.route("/<entidade>/baixar")
@login_required
def baixar(entidade):
    _validar_entidade_ou_404(entidade)
    nome_arquivo, conteudo = gerar_csv(entidade)

    registrar_auditoria(
        usuario=usuario_atual(),
        acao="BAIXAR_RELATORIO",
        tabela=entidade,
        descricao=f"Download do relatório de {entidade} ({nome_arquivo}).",
    )
    db.session.commit()

    return Response(
        conteudo,
        mimetype="text/csv",
        headers={"Content-Disposition": f"attachment; filename={nome_arquivo}"},
    )
```

app/routes/relatorios.py - a partir da linha 78

```
@relatorios_bp.route("/<entidade>/exportar-s3", methods=["POST"])
@roles_required(*PERFIS_QUE_EXPORTAM)
def exportar_s3(entidade):
    _validar_entidade_ou_404(entidade)

    if not s3_configurado():
        flash("A exportação para a nuvem não está configurada neste ambiente.", "warning")
        return redirect(url_for("relatorios.index"))

    _, conteudo = gerar_csv(entidade)
    try:
        chave = enviar_relatorio(conteudo, entidade)
        link = gerar_url_download(chave)
    except ArmazenamentoError as erro:
        flash(str(erro), "danger")
        return redirect(url_for("relatorios.index"))

    registrar_auditoria(
        usuario=usuario_atual(),
        acao="EXPORTAR_RELATORIO",
```

```
        tabela=entidade,
        descricao=f"Relatório de {entidade} exportado para o armazenamento em nuvem (objeto {chave}).",
    )
    db.session.commit()

    flash(f"Relatório enviado à nuvem. Link temporário de download: {link}", "success")
    return redirect(url_for("relatorios.index"))
```

• **/relatorios** (index): qualquer usuário logado vê a tela; o botão de exportar só aparece com permissão e S3 configurado.

• **baixar**: qualquer usuário logado baixa o CSV (os mesmos dados já são visíveis nas listagens). Gera auditoria *BAIXAR_RELATORIO*.

• **exportar_s3**: só ADMINISTRADOR e GESTAO_INFORMACAO (exportar para fora do sistema é mais sensível), método POST (protegido por CSRF). Valida a entidade (404 se fora da lista), confere se o S3 está configurado, gera, envia, cria o link e audita *EXPORTAR_RELATORIO* na mesma transação.

• A auditoria só é gravada se o envio funcionou.

Ligação com o resto do sistema: o blueprint é registrado em **app/__init__.py** e o item "Relatórios" aparece no menu lateral em **app/templates/base.html**. A tela está em **app/templates/relatorios/index.html**.

6. Correções após revisão de segurança

Uma revisão automática apontou um risco no Requisito 3, procedente. Foram feitas estas correções (todas com teste):

Problema	Correção	Onde
Uma identidade externa nova com o mesmo e-mail de uma conta já vinculada a outro identificador poderia assumir essa conta	Nunca re-vincular por e-mail se a conta já tem outro identificador externo; ignorar e relatar	diretorio_service._aplicar_usuario (parte A)
Login OIDC aceitava e-mail não confirmado pelo provedor (só recusava quando vinha false)	Exigir email_verified igual a true	oidc_service.concluir_autorizacao
Comparação do state com texto não-ASCII poderia gerar erro 500	Comparar em bytes (UTF-8) com compare_digest	auth.login_oidc_callback

Atenção na apresentação: se o provedor OIDC do cliente não enviar o claim *email_verified*, o login será recusado por segurança.

7. Testes

Arquivo: **tests/test_fase14_diretorio_relatorios.py**. Nenhum teste usa rede: LDAP com *MOCK_SYNC* do ldap3, provedor OIDC simulado (as duas funções HTTP do módulo são substituídas) e cliente S3 falso.

Como o LDAP falso é montado nos testes

`tests/test_fase14_diretorio_relatorios.py` - a partir da linha 113

```
def _conexao_ldap_mock(com_member_of=False):
    conexao = Connection(Server("ldap-falso"), user="cn=svc,o=oris", password="x", client_strategy=MOCK_SYNC)
    add = conexao.strategy.add_entry
    add("cn=svc,o=oris", {"userPassword": "x", "objectClass": ["person"], "uid": "svc", "cn": "svc"})
    add("uid=ana,ou=pessoas,o=oris", {"objectClass": ["person"], "uid": "ana", "cn": "Ana Admin", "mail":
        "ana@rede.gov"})
    add("uid=bia,ou=pessoas,o=oris", {"objectClass": ["person"], "uid": "bia", "cn": "Bia Gestora", "mail":
        "bia@rede.gov"})
    add("uid=caio,ou=pessoas,o=oris", {"objectClass": ["person"], "uid": "caio", "cn": "Caio Sem Grupo", "mail":
        "caio@rede.gov"})
    add("cn=oris-admin,ou=grupos,o=oris", {"objectClass": ["groupOfNames"], "cn": "oris-admin", "member":
        ["uid=ana,ou=pessoas,o=oris"]})
    add(
        "cn=oris-gestao,ou=grupos,o=oris",
        {"objectClass": ["groupOfNames"], "cn": "oris-gestao", "member": ["uid=bia,ou=pessoas,o=oris",
            "uid=ana,ou=pessoas,o=oris"]},
```

```
    )
    add("cn=outro,ou=grupos,o=oris", {"objectClass": ["groupOfNames"], "cn": "outro", "member":
        ["uid=caio,ou=pessoas,o=oris"]})
    conexao.bind()
    return conexao
```

Cria um servidor LDAP em memória com uma conta de serviço, três pessoas e três grupos, e liga a conexão: assim o código real de leitura é exercitado sem servidor.

Como o S3 falso é montado

`tests/test_fase14_diretorio_relatorios.py` - a partir da linha 518

```
class ClienteS3Falso:
    def __init__(self, falhar=False):
        self.objetos = {}
        self.falhar = falhar

    def put_object(self, **kwargs):
        if self.falhar:
            raise RuntimeError("AccessDenied: bucket secreto-interno")
        self.objetos[kwargs["Key"]] = kwargs

    def generate_presigned_url(self, operacao, Params, ExpiresIn):
        return f"https://s3.exemplo.test/{Params['Bucket']}/{Params['Key']}?expira={ExpiresIn}"
```

Guarda em memória o que seria enviado, para os testes conferirem bucket, chave, criptografia AES256, tipo e conteúdo.

O que os testes provam

Tema	Testes (resumo)
Mapa grupo-perfil	carrega mapa; recusa perfil inexistente e JSON inválido; maior privilégio vence
LDAP	lê usuários e grupos; sincroniza e popula usuários/grupos; é idempotente; atualiza perfil e desativa ausente; quem perde o grupo é desativado; TLS obrigatório; sem URI recusado; comando CLI registrado
Regras de segurança	não altera conta LOCAL; protege o último admin; modo simular não grava; mapa vazio recusado; conta externa não entra com senha local; não re-vincula por e-mail
OIDC	botão só com OIDC ativo; 404 se desligado; redirecionamento com PKCE/state/nonce; login completo provisiona usuário e grupos e abre sessão; rejeita state errado, nonce errado, emissor e audiência errados, usuário sem grupo, conta local, sub divergente, e-mail não verificado, endpoint sem HTTPS, callback sem iniciar
Relatórios	sanitização de fórmulas; CSV com BOM, cabeçalho e dados; equipamentos trazem unidade e serviço; download exige login, funciona para todos os perfis e é auditado; entidade inválida dá 404
S3	envio com AES256, chave e conteúdo corretos, link com validade e auditoria; GESTOR e RESPONSAVEL não exportam (403); sem login redireciona; sem bucket não envia; falha do S3 não vaza detalhes nem audita; chave sem texto do usuário; botão só aparece com permissão

Resultado da execução completa do projeto (todos os testes, antigos e novos):

```
443 passed, 2 warnings in 396.23s (0:06:36)

venv\Scripts\python.exe -m pytest -q                                    (suite completa)
venv\Scripts\python.exe -m pytest tests/test_fase14_diretorio_relatorios.py -q  (so a fase 14)
```

8. Roteiro de demonstração para o professor

A. Arquitetura e RBAC

• Abrir docs/ARQUITETURA.md: papéis, recursos, matriz e diagrama.

• Mostrar roles_required em app/utils/decorators.py e uma rota protegida.

• Entrar como GESTOR e tentar abrir /usuarios: aparece "Acesso negado" (403).

B. LDAP

```
1) No .env: LDAP_URI=ldaps://..., LDAP_BIND_DN, LDAP_BIND_PASSWORD,
   LDAP_USER_BASE, LDAP_GROUP_BASE e DIRETORIO_MAPA_GRUPO_PERFIL
2) flask sincronizar-ldap --simular      (mostra o que mudaria)
3) flask sincronizar-ldap                (grava)
4) Tela Usuarios: aparecem contas com origem LDAP e perfis conforme os grupos
5) Tela Auditoria: CRIAR_USUARIO / ALTERAR_PERFIL / DESATIVAR_USUARIO
Sem servidor LDAP real, mostrar os testes passando (usam LDAP falso).
```

C. OIDC

```
1) No .env: OIDC_ENABLED=true, OIDC_ISSUER=https://..., OIDC_CLIENT_ID,
   OIDC_CLIENT_SECRET, DIRETORIO_MAPA_GRUPO_PERFIL
2) Abrir /login: aparece o botao "Entrar com login institucional (SSO)"
3) Clicar, autenticar no provedor, voltar logado com o perfil do grupo
4) Auditoria: "Login via OIDC".
```

D. Exportação ao S3

```
1) Definir S3_BUCKET (e credenciais AWS no ambiente, ou IAM Role)
2) Entrar como ADMINISTRADOR ou GESTAO_INFORMACAO -> menu Relatorios
3) "Baixar CSV" (download local) e "Exportar para S3" (envia e mostra link)
4) Como GESTOR: o botao nao aparece e a rota devolve 403
5) Tela Auditoria: BAIXAR_RELATORIO e EXPORTAR_RELATORIO
```

Para a banca: LDAP, OIDC e S3 reais exigem o servidor de diretório, o provedor de identidade e o bucket; no projeto tudo é configurável por variáveis de ambiente e nenhum segredo fica versionado.

9. Mapa de arquivos criados e alterados

Arquivo	Papel
app/models/grupo.py (novo)	Grupo e tabela usuario_grupos
app/models/usuario.py, enums.py, __init__.py	origem, identificador_externo, grupos, OrigemDiretorio
app/services/diretorio_service.py (novo)	Regras de sincronização e mapa grupo-perfil
app/services/ldap_service.py (novo)	Leitura do LDAP
app/services/oidc_service.py (novo)	Fluxo OIDC com PKCE
app/services/relatorio_service.py (novo)	CSV e sanitização
app/services/armazenamento_service.py (novo)	Envio ao S3 e URL pré-assinada
app/routes/auth.py	Rotas SSO; senha só para contas LOCAL
app/routes/relatorios.py e templates/relatorios/ (novos)	Telas e RBAC dos relatórios
app/cli.py	flask sincronizar-ldap
app/__init__.py, templates/base.html, templates/login.html	Registro do blueprint, menu, botão SSO
config.py, .env.example, requirements.txt	Configuração e dependências (ldap3, boto3)
database/schema.sql, database/migrations/0002_*.sql	Esquema e migração
docs/ARQUITETURA.md, docs/SEGURANCA.md, README.md	Documentação
tests/test_fase14_diretorio_relatorios.py (novo)	Testes da fase 14